CampusOS API Quick Guide

A short version of the full CampusOS API Guide, made for printing. The full guide has every field, response and error code.

What is an API?

An **API** (Application Programming Interface) is how your frontend talks to the server. Your app sends a **request**, and the server sends back a **response**. The response is usually **JSON** (text made of "key": value pairs).

Every request has these parts:

Part	What it is	Example
Method	What you want to do	`GET` read · `POST` create · `PATCH` edit some fields · `PUT` set or replace · `DELETE` remove
URL	Base URL + endpoint path	`{baseUrl}/api/v1/fixit/tickets`
Path parameter	An id inside the path	`/tickets/{ticket_id}` → `/tickets/TKT-8206...`
Query parameter	Filters after ?	`/tickets?status=open&limit=25`
Headers	Extra info, such as who you are	`Authorization: Bearer <token>`
Body	The data you send (POST / PATCH / PUT)	`{ "title": "Fan not working" }`

The body comes in one of two formats:

- **JSON** (`Content-Type: application/json`) for plain data.
- **Form-data** (`multipart/form-data`) when you upload a file, like a form with fields and an attachment.

The server answers with a **status code**:

Code	Meaning
`200` / `201`	Success / created
`400` / `422`	Something is wrong with what you sent
`401`	Not logged in, or the token expired
`403`	Logged in, but not allowed to do this
`404`	Not found (or not yours)
`409`	Not possible right now (for example, the item is already closed)
`429`	Too many requests: wait and try again

In JavaScript, a call looks like this:

```
      const res = await fetch(`${baseUrl}/api/v1/fixit/tickets?status=open`, { headers: { Aut
horization: `Bearer ${token}` }, }); const data = await res.json(); // { items: [...], count,
next_cursor }
```

You can try every endpoint in the browser at **`{baseUrl}/docs`**.

Getting started

1. **Admin login:** `POST /api/v1/auth/admin/login` with `{ "email_id", "otp" }` from the organisers.
2. **Create users** (as admin): `POST /api/v1/admin/users`. The response includes each user's otp.

Office staff dept	Runs
Student Affairs	DocDesk (document requests)
Campus Maintenance	FixIt (maintenance tickets)

Create-user bodies (role decides the fields; all are required):

```
      { "role": "student", "name": "Asha Kumar", "usn": "1CF24CS001", "dept": "CSE", "email_i
d": "asha@team1.com", "phone_number": "9876543210" } { "role": "faculty", "name": "Dr. Meera
Rao", "designation": "Professor", "email_id": "meera@team1.com", "phone_number": "9876543212"
} { "role": "office_staff", "name": "Ravi Shetty", "dept": "Student Affairs", "designation":
"Clerk", "email_id": "ravi@team1.com", "phone_number": "9876543211" }
```

The `email_id` must use your team's domain.

Sample values:

What	Examples
Student dept	CSE, ISE, ECE, MBA
Office dept	Student Affairs, Campus Maintenance, Accounts, Library
Document types	Bonafide Certificate, Transcript, Scholarship Form, Experience Certificate
Issue types	Electrical, Plumbing, Furniture, IT & Network

Good to know

- **Lists** return `{ "items": [...], "count", "next_cursor" }`. For the next page, send `?cursor=<next_cursor>`. `null` means it was the last page.
- **PATCH** changes only the fields you send. For records with a file, a new file replaces the old one, and `remove_<name>=true` deletes it.
- **Errors** look like `{ "error": { "code", "message", "details" } }`. Base your UI logic on `error.code`.
- **File links** (url) expire after about 15 minutes. Download endpoints always work.
- **Dates** are `YYYY-MM-DD`. Response times are in UTC.

Other account endpoints:

Method	Endpoint	Who	What
`GET`	`/api/v1/users/me`	users	My profile
`GET`	`/api/v1/admin/me`	admin	Admin profile
`GET`	`/api/v1/admin/users?role=`	admin	List users (with OTPs)
`GET / PATCH / DELETE`	`/api/v1/admin/users/{email_id}`	admin	View, edit or delete a user
`POST`	`/api/v1/admin/users/{email_id}/regenerate-otp`	admin	New OTP

1. FindIt: Lost & Found · `/api/v1/findit`

Office staff post found items, owners claim them, and staff resolve the post by approving one claim.

Method	Endpoint	Who	What

Method	Endpoint	Who	What
DELETE	/claims/{id}	claimant	Withdraw while pending
POST / PATCH / DELETE	/posts/{id}/resolution	office staff	Resolve { "claim_id", "note" } / change / undo
★ POST	/lost-requests	non-admin	Report a lost item: { "title"*, "description", "location_lost", "lost_on" }
★ GET	/lost-requests?status=	non-admin	Staff see everyone's, others see their own
★ PATCH / DELETE	/lost-requests/{id}	owner	Edit, or set "status": "archived" once found
★ PATCH	/posts/{id}	office staff	Archive / reopen: send status=archived or status=active. List with GET /posts?status=archived

2. Notice Hub: Campus notices · /api/v1/noticehub

Admin, faculty and office staff post notices. Readers ask doubts, and the publisher answers each doubt once.

Method	Endpoint	Who	What
POST	/notices	admin, faculty, staff	Form-data: title*, body*, ★ pinned, ★ publish_at, ★ attachment
GET	/notices · /notices/{id}	everyone	List active notices / view one
PATCH / DELETE	/notices/{id}	publisher (admin can delete)	Edit / delete
POST	/notices/{id}/doubts	non-admin	Ask: { "question": "5+ chars" }
GET	/notices/{id}/doubts?answered=false	everyone	Doubts with responses
PATCH / DELETE	/doubts/{id}	asker	Edit (until answered) / delete
POST / PATCH / DELETE	/doubts/{id}/response	publisher, admin	Answer { "message" } / edit / remove
★ POST / PATCH	/notices · /notices/{id}	publisher	**Schedule:** send publish_at=2026-11-03T09:00:00 (IST). It goes live automatically. List with GET /notices?status=scheduled
★ PATCH	/notices/{id}	publisher	**Archive / restore:** send status=archived or status=active. List with GET /notices?status=archived
★ POST / PATCH	/notices · /notices/{id}	publisher	**Pin / unpin:** send pinned=true or pinned=false. Filter with GET /notices?pinned=true
★ GET	/notices/{id}/attachment	everyone	**Download** the attachment (PDF/JPEG/PNG, 10 MB)

3. DocDesk: Document requests · /api/v1/docdesk

Method	Endpoint	Who	What
★ GET	/doc-types?status= · /doc-types/{id}	everyone	Fetch document types (for the dropdown)
★ POST / PATCH / DELETE	/doc-types · /doc-types/{id}	SA	Create { "name", "description" }, edit, archive ("status": "archived"), delete
★ PUT	/requests/{id}/tat	SA	Turnaround time: { "expected_by": "2026-11-05", "note" }
★ POST	/requests/bulk-tat	SA	Bulk: { "request_ids": [...], "expected_by", "note" }. All or nothing
★ GET	/requests/{id}/document	requester, SA, admin	Download the document sent with the answer
★ PUT / GET / DELETE	/requests/{id}/filled-form	requester (SA can read)	Upload the filled-in form (form-data file) / download / remove

4. FixIt: Maintenance tickets · /api/v1/fixit

Users raise tickets, and Campus Maintenance (CM) fix them. Status: open ⇄ resolved → acknowledged.

Method	Endpoint	Who	What
POST	/tickets	non-admin	Form-data: issue_type_id*, title*, description*, photo
GET	/tickets?status=&issue_type_id=	everyone	CM and admin see all, others see their own
PATCH / DELETE	/tickets/{id}	reporter while open (CM / admin can delete any)	Edit (photo, remove_photo=true) / delete
GET	/tickets/{id}/photo	reporter, CM, admin	Download the photo
PUT	/tickets/{id}/status	CM	{ "status": "open" or "resolved", "note" }
POST	/tickets/{id}/acknowledge	reporter	Confirm it's fixed and close the ticket
★ PUT	/tickets/{id}/tat	CM	Turnaround time: { "expected_by", "note" }. The ticket shows overdue: true when the date passes
★ POST / PATCH / DELETE	/issue-types · /issue-types/{id}	CM	Issue types CRUD: { "name", "description" }
★ GET	/issue-types · /issue-types/{id}	everyone	Fetch issue types for the "Raise a ticket" dropdown

* = required field · ★ = bonus · "non-admin" = student, faculty and office staff